

Present Value Engine Implementation Walkthrough

How the Go port values streams of money across time: field-presence classification, the geometric-series summation, the seven backward solvers, COLA timing, variable-rate discounting, and the life-contingency fold-in.

Scope. The Go PV package `internal/finance/presentvalue/` (`calc.go`, `backward.go`, `variablerate.go`, `types.go`, `advisories.go`), the API wiring in `internal/api/handlers.go`, and the shared math in `internal/finance/interest/` and `internal/dateutil/`.

Audience. Engineers maintaining or extending the Go port. Every claim is anchored to a `file:line` reference under `internal/`.

Lineage. A faithful port of `PRESVALU.pas` / `PVLXSCRN.pas`; for the original algorithm see the companion `legacy/code_docs/present_value_dos_walkthrough.pdf`. The life-contingency integration is detailed in the actuarial Go walk-through; here it is covered where it hooks into the PV sums.

Validation. The engine is checked to the cent against the real DOS PV engine via a headless differential oracle across roughly **11,900 worksheets** plus about **2,400 per-row value checks** (the oracle now emits each row's individual present value, not just the screen total).

Contents

0 How to read this document

1 Entry & data model

2 Classification & dispatch

3 The summation mathematics

3.1 The geometric kernel · 3.2 PeriodicSummation · 3.3 COLA modes

4 Forward calc & the rate / as-of solves

5 Backward solvers

6 The life-contingency fold-in

7 The variable-rate engine

8 Math primitives & boundary cases

9 A worked trace

10 Appendix: call graph & file:line index

0 How to read this document

The Go PV engine is a worksheet that values streams of money. It has three editable blocks — lump-sum single payments, periodic streams (with optional COLA escalation), and a rate / present-value line carrying the discount rate and the as-of date. The caller leaves one cell blank and the engine solves for it. A *variable-rate* mode

replaces the single discount rate with a schedule of rates over time, and a life-contingency mode weights each payment by a survival probability.

Citation convention

References look like `calc.go:300` and resolve under `internal/finance/presentvalue/` unless another path is named (e.g. `api/handlers.go:1317`). Pascal lineage is noted as `PRESVALU.pas:517`.

The one formula to know. A periodic stream's PV is $\text{amt} \times \text{PeriodicSummation}$, and that factor is a geometric series $(1-f^n)/(1-f)$ whose ratio $f = e^{(\text{cola}-\text{rate})/P}$ nets escalation against discounting. When $\text{cola} = \text{rate}$, $f \rightarrow 1$ and the series collapses to n — the load-bearing edge case.

1 Entry & data model

The HTTP entry is `HandlePVCalc` `api/handlers.go:1317`, decoding a `PVRequest` `api/handlers.go:344` whose optional scalars are pointers — omit `Rate` or `AsOfDate` to solve for it; supply `SumValue` to enable backward mode. Each non-nil pointer stamps a status; a nil pointer leaves `StatusEmpty`. Two subtleties:

- The as-of date sets `InOutInput` only when present `api/handlers.go:1347–1357`; the rate is stamped `StatusFromRate` `api/handlers.go:1361–1366`; `SumValue` presence flips the screen into backward mode `api/handlers.go:1369–1372`.
- A non-empty `RateSchedule` switches the engine into variable-rate mode `api/handlers.go:1475–1496`.

The handler then calls `Calculate(input)` `api/handlers.go:1499`.

The model lives in `types.go`: `RateEntry` (the stored continuous rate + status) `types.go:30`, `LumpSumPayment` (date / amount / value + contingency) `types.go:38`, `PeriodicPayment` (from / to / per-yr / amount / COLA / value, plus the per-payment `Installments` used under a life contingency) `types.go:52`, `PresValLine` (as-of, the rate entry, and the total `SumValue`) `types.go:93`, `PVSettings` (basis, per-yr, COLA-month, exact flag, day-count) `types.go:105`, `RateLine` (the variable-rate schedule: date + continuous rate) `types.go:124`, and `PVInput` with its optional `Actuarial *ActuarialConfig` `types.go:130–135`. Status codes `types/constants.go:41–58` use the $\geq$ `InOutDefault` (2) = "known" rule; a per-row line-status code `types/constants.go:182–183` (`LineFullySpecified=254`, `LineContainsUnknown=253`) is computed by counting present fields.

2 Classification & dispatch

`Calculate` `calc.go:300` runs three pre-dispatch checks — a two-life-contingency guard `calc.go:307` (reject a two-life contingency with no second life table, naming the offending row), the variable-rate branch `calc.go:314` (if `RateSchedule` is set, `FirstPass` is skipped entirely), and an unknown-POD shortcut `calc.go:339` — then `FirstPass` `backward.go:163` classifies each block, assigning a line-status and resolving the `Frontward/Backward` flags.

How field-presence picks a path. A nil JSON pointer became `StatusEmpty` at the API boundary, so by the time `FirstPass` runs, "blank" is simply `"status < InOutDefault"`. `FirstPass` counts the present core fields of every row to assign its line-status, deletes empty rows, then sets two flags. `Frontward` is true only when rate and as-of are present and every populated row is fully specified. `Backward` fires when a row-level target exists (or the screen `SumValue` with a missing rate/as-of), and it selects a `BackwardKind` by *which* field is blank:

Blank field	BackwardKind	Row / path	Set at
lump amount	PV-1 BackwardLumpAmount	date + value given	backward.go:438
lump date	PV-2 BackwardLumpDate	amount + value given	backward.go:444
periodic amount	PV-4 BackwardPeriodicAmount	both dates + value	backward.go:467
periodic to-date	PV-5 BackwardPeriodicToDate	from + amount + value	backward.go:475
periodic from-date	PV-6 BackwardPeriodicFrom	to + amount + value	backward.go:477
rate	PV-8 BackwardRate	asof + rows + SumValue	backward.go:500
as-of date	PV-9 BackwardAsOf	rate + rows + SumValue	backward.go:507

The BackwardKind enum is declared at `backward.go:140–151`. The dispatch decision `calc.go:348–362`: both flags → "more than one missing field" error; Backward → BackwardCalc (a switch on BackwardKind at `backward.go:541–556`); !Forward → insufficient-input error; else forwardOnly. (PV-3 and PV-7 are intentionally absent — they correspond to dead DOS `V_3` code.)

3 The summation mathematics

Throughout: r = discount rate, c = COLA, P = RealPerYr(peryr), n = installments. `YearsDif(asof, date)` is negative for future payments, so the exponential discounts.

3.1 The geometric kernel

`SumFormula(lnf, n)` `calc.go:20` is the closed form of $\sum_{i=0}^{n-1} e^{i \cdot \ln f}$:

$$\text{SumFormula} = (1 - e^{(n \cdot \ln f)}) / (1 - e^{(\ln f)}), \quad \rightarrow n \quad \text{when } |\ln f| < \text{teeny}$$

The teeny (1e-10) cutoff collapses the series to n equal terms `calc.go:21–24`; a tiny (1e-5) band swaps in 2nd-order Taylor expansions of $1 - e^x$ to avoid catastrophic cancellation near zero `calc.go:26–47`.

3.2 PeriodicSummation

`PeriodicSummation` `calc.go:93` sets the per-period log-ratio $\ln f = (cola - rate)/P$ `calc.go:97` (so $f = e^{\ln f}$), guards an infinite series ($\ln f \geq 0$ with a "forever" to-date → error) `calc.go:100–103`, then returns the closed form:

$$\text{PeriodicSummation} = e^{(r \cdot \text{since})} \cdot \text{SumFormula}(\ln f, n) \cdot [\text{phase factor}]$$

where `since` measures from the as-of date to the stream's reference point. The branch chooses whether to measure from a "standard loan date" one period before the first payment ($\text{asof} \leq \text{fromDate}$, with an $e^{-r/P}$ phase correction) or from the to-date (accumulating forward, with a COLA range adjustment) `calc.go:162–208`. The final $e^{r \cdot \text{since}}$ discount is applied at `calc.go:204–209`.

The cola = rate degeneracy. When $c = r$, $\ln f = 0$, $f = 1$, and the geometric ratio is 0/0. The teeny branch collapses it to n equal terms `calc.go:162–164`: every escalated payment discounts to the same value, so the stream is worth n payment PVs. Every backward solver special-cases this.

3.3 COLA modes

COLA is entered as a *yield*. Three policies are selected by `settings.COLAMonth`:

- **Continuous** (`COLAContinuous`): convert the yield to a continuous rate $\text{cola} = \log(1 + \text{cola})$ and fold it into `lnf` so the closed-form formula applies `calc.go:124–129`.
- **Stepped** (anniversary or a specific calendar month 1–12): `periodicSumAnnualCOLA` `calc.go:246` applies the $(1 + \text{cola})$ multiplier once per anniversary instead of smoothly each period; `firstCOLAStepDate` `calc.go:230` finds the first step. The two paths are kept consistent so they agree at integer years.
- **Exact**: a period-by-period walk `calc.go:131–153`, used for weekly/biweekly, life-contingent, or explicitly-exact streams.

The stepped path is the DOS default (the closed-form continuous formula over-counts per-payment growth); the gate at `calc.go:118–122` routes `peryr>1` COLA streams there unless continuous mode is selected.

4 Forward calc & the rate / as-of solves

`forwardOnly` `calc.go:467` requires rate + as-of present. A lump value is $\text{amt} \cdot e^{\text{rate} \cdot \text{YearsDif}(\text{asof}, \text{date})}$ via `LumpSumValue` `calc.go:65`; a periodic value is $\text{amt} \cdot \text{PeriodicSummation}$ (or the actuarial exact path); `sumValue` aggregates all rows `calc.go:483–565`.

Unknown rate (PV-8) `solveRate` `backward.go:1196`: a damped secant on the PV residual. The Newton/secant step is $\text{diff} = (\text{target} - \text{sum}) \cdot \text{diff} / (\text{sum} - \text{oldSum})$, clamped to ± 0.04 , first guess 0.1, with a second-pass restart from 0, capped at 30 iterations to $|\text{diff}| < \text{teeny}$. (Ported from `PRESVALU.pas:693–754`.)

Unknown as-of date (PV-9) `solveAsOf` `backward.go:1276`: a near-closed-form fixed point $\text{asof} += \ln(\text{target} / \text{sum}) / \text{rate}$, ≤ 10 iterations to < 0.002 years. It seeds the search at 2000-01-01 to mirror the DOS year-byte seed and dodge the `AddYears` 128-year guard `backward.go:1294–1300`. Both share `evaluatePVAt` `backward.go:1362`, the non-mutating forward valuation kernel (which includes the POD term).

5 Backward solvers

Every row solver first derives a target by subtracting the known rows (and the POD) from `SumValue` via `computeKnownRowSum` `backward.go:584`.

Path	Solve	Method	Line
PV-1	lump amount	direct algebraic inverse: $\text{amount} = \text{val} \cdot e^{\text{rate} \cdot \text{YearsDif}(\text{date}, \text{asof})}$	<code>backward.go:644</code>
PV-2	lump date	Newton on $\text{amt} \cdot e^{\text{rate} \cdot \text{yrs}}$; ≤ 30 iters. Rejected for life-contingent rows (survival depends on the unknown date)	<code>backward.go:709</code>
PV-4	periodic amount	direct: $\text{amount} = \text{val} / \text{weightedPeriodicFactor}$	<code>backward.go:820</code>
PV-5 / PV-6	periodic to / from date	analytic first estimate, then <code>refinePeriodicDate</code> snaps to a true installment (To-Date on the period grid; From-Date bisected on the calendar day)	<code>backward.go:891</code>

The date solvers reuse the exact forward factor (`weightedPeriodicFactor` `backward.go:92`, `evaluatePVAt`) that they invert, so round-trips hold. The refinement helpers are `refinePeriodicDate` `backward.go:1061` and

`refineDateByDays backward.go:1141` ; a one-cent amount correction absorbs the residual of forcing a whole-period count.

6 The life-contingency fold-in

When a row carries a contingency code and `input.Actuarial` is set, each row's value is weighted by a survival probability, and a Payment-on-Death term is folded into the total. This mirrors the DOS `fold_in_life` branches in `PRESVALU.pas`.

- **Lump rows.** `val *= LifeProb(date, act)` and the realized probability is stashed on the row `calc.go:495–501` (ported from `PRESVALU.pas:313 / PRESVALU.pas:884`).
- **Periodic rows.** When contingent, the engine forces the exact period-by-period summation and multiplies each payment by `LifeProb(t, act)` via `periodicWithActuarial calc.go:535–542, 587` (ported from the `fold_in_life` per-payment loop at `PRESVALU.pas:541–560`). The per-payment breakdown is returned as `Installments` so the PV table can print date / if-paid value / probability / weighted value.
- **Payment-on-Death.** After all rows, `PODValue(asOf, rate)` is added to `sumValue` and echoed on the result `calc.go:558–562` (ported from `PRESVALU.pas:896`). When the POD amount itself is blank, `solveUnknownPOD calc.go:419` back-solves it: the POD's PV is linear in the POD amount, so it computes the PV of everything else, probes a unit death benefit, and divides — the closed-form mirror of DOS `ComputeUnknownPOD`.

The recovered 1988 HHS life tables feed in through `ActuarialConfig` (its `Table1 / Table2` fields, with dates of birth), supplied by the API alongside the PV request. With `Actuarial == nil` every contingency is inert and no weighting is applied; the two-life guard in §2 prevents a silent `s2 = 1` degeneration when a two-life contingency is requested without a second table.

7 The variable-rate engine

A rate schedule replaces a single scalar r with a piecewise-constant function; the discount becomes $\int r(s) ds$ rather than `rate·YearsDif`. `integrateRateForward variableRate.go:73` walks time, accruing `rate·YearsDif` within each band (the first entry's rate runs from $-\infty$). `VRDiscountFactor variableRate.go:124` is the per-payment multiplier:

```
paymentDate > asof: e^(-∫_asof^pay r)   (discount)
paymentDate < asof: e^(+∫_pay^asof r)   (accumulate)
```

`vrPeriodicValue variableRate.go:159` walks each installment applying COLA and the discount factor (and a `LifeProb` weight under a contingency). `forwardVariableRate variableRate.go:273` is the entry; it requires every row fully specified (no per-row closed-form backward solve — DOS: "rates cannot be the target"). The supported VR backward solves exploit linearity / monotonicity: `solveVariableRateAmount variableRate.go:434` and `solveVariableRatePOD variableRate.go:495` probe two forward runs and divide; `solveVariableRateDate variableRate.go:582` brackets a sign change and bisection on the calendar day against the true VR valuation. Variable-rate mode never uses the closed-form `SumFormula` — it always walks period-by-period and integrates the schedule.

8 Math primitives & boundary cases

Exxp / Lnn `interest/math.go:43` / `interest/math.go:67` (safe exp/ln; overflow at $x > 70$ is the "infinite present value" guard, underflow at $x < -70$ returns $1e-32$, with Taylor series near the origin), RealPerYr / YieldFromRate / RateFromYield `interest/rates.go:42` / `interest/rates.go:63` / `interest/rates.go:79`, and YearsDif / AddPeriod / AddYears `dateutil/dateutil.go:643` / `dateutil/dateutil.go:442` / `dateutil/dateutil.go:383`.

Boundary guards. teeny degeneracy/convergence cutoffs; an explicit infinite-series error when $\ln f \geq 0$ with a "forever" to-date `calc.go:100`; the AddYears $|yrs| > 128$ cap that backstops a runaway date solve `dateutil/dateutil.go:383`; and per-solver iteration caps (lump-date 30, rate 30×2 , as-of 10, date refinement).

9 A worked trace

Forward. One periodic stream of \$1,000/month for 10 years, 5% rate, no COLA, value blank — solve the value.

#	What happens	Code
1	JSON omits the row's value → it stays <code>StatusEmpty</code> ; rate and <code>asOfDate</code> present.	<code>api/handlers.go:1361</code>
2	<code>Calculate</code> passes the pre-dispatch guards (no second life, no schedule, no unknown POD) and calls <code>FirstPass</code> .	<code>calc.go:300</code>
3	<code>FirstPass</code> : the row is fully specified (amount present, value blank but value isn't a <i>core</i> field), rate + as-of present → <code>Forward = true</code> , <code>Backward = false</code> .	<code>backward.go:163</code>
4	Dispatch falls through to <code>forwardOnly</code> ; <code>NInstallments = 120</code> ; <code>factor = PeriodicSummation(...)</code> with $\ln f = (0 - 0.05)/12 < 0$ so the geometric sum is finite; <code>pp.Val = 1000 * factor</code> .	<code>calc.go:507–551</code>
5	<code>sumValue</code> aggregates; the result echoes the rate and as-of date used.	<code>calc.go:565</code>

Backward (PV-4). Now blank the row *amount* instead and supply the screen `SumValue`. The API sets `SumValueStatus = InOutInput`; `FirstPass` sees a periodic row with both dates + value but no amount, so it sets `BackwardKind = BackwardPeriodicAmount` `backward.go:467`. `BackwardCalc` derives the row target by subtracting the known rows (and POD) via `computeKnownRowSum`, then `solvePeriodicAmount` divides: `amount = target / weightedPeriodicFactor` `backward.go:820` — the exact inverse of the forward factor, so the forward → backward round-trip recovers \$1,000.

The degeneracy. Set `cola = 0.05` equal to the rate: $\ln f = 0$, the summation flattens to $n = 120$, and the stream is worth exactly 120 discounted payments. The teeny branch of `PeriodicSummation` `calc.go:162` takes over with no division by $(1-f)$, and `solvePeriodicAmount` divides by the same n -term factor — the round-trip still holds.

10 Appendix: call graph & file:line index

HandlePVCalc	api/handlers.go:1317
└ build PVInput (pointer → status) → Calculate	calc.go:300
└─┬ checkSecondLifeProvided	calc.go:387
└─┬ [RateSchedule ≠ ∅] variable-rate branch	calc.go:314
└─┬┬ solveVariableRate POD/Amount/Date	variablerate.go:495 / 434 / 582
└─┬┬┬ forwardVariableRate	variablerate.go:273
└─┬┬┬┬ VRDiscountFactor → integrateRateForward	variablerate.go:124 / 73
└─┬ [PODUnknown] solveUnknownPOD	calc.go:419
└─┬ FirstPass (classify; Frontward/Backward + Kind) backward.go:163	
└─┬┬ Backward → BackwardCalc	backward.go:522
└─┬┬┬ computeKnownRowSum (target - known - POD)	backward.go:584
└─┬┬┬┬ PV-1/2 lump amount/date	backward.go:644 / 709
└─┬┬┬┬ PV-4 periodic amount	backward.go:820
└─┬┬┬┬ PV-5/6 periodic dates → refinePeriodicDate	backward.go:891 / 1061
└─┬┬┬┬ PV-8 rate (secant) → evaluatePVAt	backward.go:1196 / 1362
└─┬┬┬┬ PV-9 as-of (fixed point)	backward.go:1276
└─┬ forwardOnly	calc.go:467
└─┬┬ LumpSumValue $\text{amt} \cdot e^{\{\text{rate} \cdot \text{YearsDif}\}}$	calc.go:65
└─┬┬ PeriodicSummation → SumFormula annual-COLA exact loop	calc.go:93 / 20 / 246
└─┬┬ LifeProb fold-in (lump/periodic)	calc.go:497 / 535
└─┬┬ PODValue add	calc.go:559

Symbol	Reference
HandlePVCalc / Calculate	api/handlers.go:1317 · calc.go:300
FirstPass / BackwardCalc	backward.go:163 / backward.go:522
SumFormula / PeriodicSummation / annual COLA	calc.go:20 / calc.go:93 / calc.go:246
forwardOnly / evaluatePVAt	calc.go:467 · backward.go:1362
LifeProb fold-in / PODValue / solveUnknownPOD	calc.go:497 · calc.go:559 · calc.go:419
Variable-rate: forward / discount / integrate	variablerate.go:273 / variablerate.go:124 / variablerate.go:73
Math primitives (exp/lr, rates, dates)	interest/math.go:43 · interest/rates.go:42 · dateutil/dateutil.go:643

Per%Sense DOS → Go port. The life-contingency integration is documented in [actuarial_go_walkthrough.pdf](#).
 Companions: [amortization_go_walkthrough.pdf](#), [mortgage_go_walkthrough.pdf](#).